

# DESIGN DOCUMENT

BizFlow – A Multi-Tenant Distributor Inventory, Billing and Business Management System

## 1. INTRODUCTION

The design phase of software development transforms user requirements into a technical blueprint that guides implementation. It defines the architecture, modules, database structure, workflows, interfaces, and interactions between system components.

BizFlow Version 2 is designed as a modern multi-tenant web application that enables distributors and small businesses to manage inventory, invoices, payments, and business records through a centralized platform. The design focuses on scalability, maintainability, security, and ease of use.

The system follows a client-server architecture where the frontend communicates with backend REST APIs, and the backend interacts with a PostgreSQL database to persist business information.

## 2. SYSTEM ARCHITECTURE

The BizFlow architecture consists of three major layers:

### A. Presentation Layer (Frontend)

Technology Stack:

- React.js
- Vite
- React Router
- React Query
- Tailwind CSS

Responsibilities:

- User Interface Rendering
- Navigation
- Form Validation
- API Requests
- State Management

### B. Application Layer (Backend)

Technology Stack:

- Node.js
- Express.js
- Sequelize ORM

Responsibilities:

- Authentication
- Business Logic
- Invoice Processing

- Inventory Updates
- Payment Processing
- Data Validation

### **C. Data Layer (Database)**

Technology:

- PostgreSQL

Responsibilities:

- Data Storage
- Data Retrieval
- Transaction Management
- Data Integrity

System Flow:

User → React Frontend → Express API → PostgreSQL Database

Database Response → API → Frontend → User

## **3. SYSTEM MODULES**

### **3.1 Authentication Module**

Purpose:

Provides secure access to the system.

Features:

- Registration
- Login
- Logout
- Session Management
- Protected Routes

Workflow:

1. User enters credentials.
2. Backend validates credentials.
3. Session is created.
4. User gains access to protected resources.

### 3.2 Tenant Management Module

Purpose:

Supports multiple organizations on the same platform.

Responsibilities:

- Tenant Creation
- Tenant Isolation
- User-Tenant Mapping

Advantages:

- Scalability
- Security
- Multi-Organization Support

### 3.3 Dashboard Module

Purpose:

Provides business insights.

Displayed Information:

- Total Invoices
- Total Sales
- Total Purchases
- Pending Payments
- Inventory Statistics

Benefits:

- Better visibility
- Quick decision-making

### 3.4 Product Catalog Module

Purpose:

Stores reusable product information.

Features:

- Add Product
- Search Product
- Delete Product
- Manage Product Data

Product Attributes:

- Name
- Brand
- HSN Code
- Unit Name
- Unit Quantity
- MRP

### 3.5 Inventory Module

Purpose:

Tracks stock availability.

Features:

- Inventory Listing
- Automatic Updates
- Search Functionality

Inventory Logic:

Stock-In Invoice:

Inventory += Quantity

Stock-Out Invoice:

Inventory -= Quantity

### 3.6 Generate Bill Module

Purpose:

Creates invoices.

Invoice Types:

- Stock-In
- Stock-Out

Process:

1. Select Invoice Type
2. Enter Party Information
3. Add Products
4. Calculate Taxes
5. Calculate Discounts
6. Generate Invoice
7. Update Inventory
8. Generate PDF

### 3.7 Payment Module

Purpose:

Tracks financial transactions.

Features:

- Record Payment
- Payment History
- Pending Balance Calculation
- Bill Status Tracking

Bill States:

- Pending
- Partial
- Paid

Formula:

Pending Amount =

Grand Total – Total Payments

### 3.8 View Bills Module

Purpose:

Provides invoice history.

Features:

- Invoice Search
- Invoice Filtering
- Invoice Details
- PDF Download

### 3.9 PDF Generation Module

Purpose:

Creates printable invoices.

Contents:

- Company Details
- Party Information
- Product Details
- Taxes
- Totals
- Payment Information

## 4. DATABASE DESIGN

The database is implemented using PostgreSQL.

### 4.1 Tenant Table

Stores organization information.

Fields:

- tenant\_id (PK)
- company\_name
- gst\_number
- phone\_number
- address
- created\_at

### 4.2 User Table

Stores user information.

Fields:

- user\_id (PK)
- tenant\_id (FK)
- email
- password
- role
- created\_at

Relationship:

One Tenant → Many Users

### 4.3 Product Catalog Table

Stores product information.

Fields:

- product\_id (PK)
- tenant\_id (FK)
- name
- brand
- hsn\_code
- unit\_name
- unit\_qty
- mrp

Relationship:

One Tenant → Many Products

#### 4.4 Inventory Table

Stores inventory quantities.

Fields:

- inventory\_id (PK)
- tenant\_id (FK)
- product\_name
- quantity
- hsn\_code
- unit\_name

Relationship:

One Tenant → Many Inventory Records

#### 4.5 Invoice Table

Stores invoice information.

Fields:

- invoice\_id (PK)
- tenant\_id (FK)
- invoice\_number
- invoice\_type
- invoice\_to
- invoice\_from
- grand\_total
- pending\_amount
- bill\_state

Relationship:

One Tenant → Many Invoices

#### 4.6 Invoice Item Table

Stores invoice products.

Fields:

- item\_id (PK)
- invoice\_id (FK)
- name
- quantity
- rate
- discount

- cgst
- sgst
- total\_amount

Relationship:

One Invoice → Many Invoice Items

#### 4.7 Payment Table

Stores payment information.

Fields:

- payment\_id (PK)
- invoice\_id (FK)
- amount
- payment\_method
- created\_at

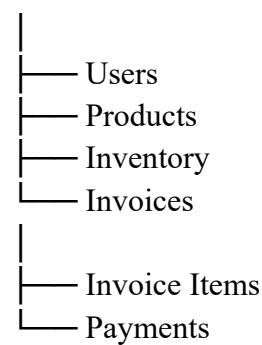
Relationship:

One Invoice → Many Payments

### 5. ENTITY RELATIONSHIP MODEL

Major Relationships:

Tenant



Relationship Summary:

1 Tenant → Many Users

1 Tenant → Many Products

1 Tenant → Many Inventory Records

1 Tenant → Many Invoices

1 Invoice → Many Invoice Items

1 Invoice → Many Payments

### 6. DATA FLOW DESIGN

Level 0 DFD



External Entity:

Distributor

Main Process:

BizFlow System

Inputs:

- Product Data
- Invoice Data
- Payment Data

Outputs:

- Inventory Updates
- Invoice PDFs
- Dashboard Reports

Level 1 DFD

Distributor



Authentication Module



Dashboard Module



Generate Bill Module



Inventory Module



Payment Module



Database

Database



View Bills Module



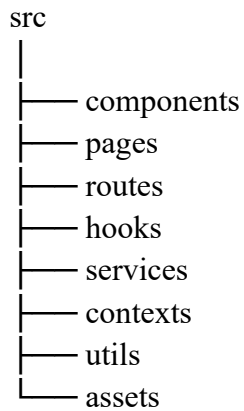
PDF Generator



Distributor

## 7. FRONTEND DESIGN

Frontend Architecture:



Major Screens:

- Login
- Register
- Dashboard
- Product Catalog
- Inventory
- Generate Bill
- View Bills

State Management:

React Query:

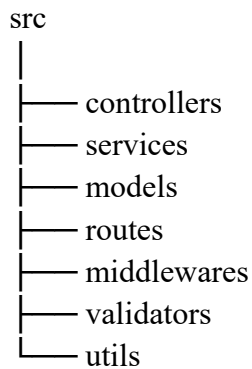
- API Data
- Caching
- Synchronization

Context API:

- Authentication State
- Temporary UI State

## 8. BACKEND DESIGN

Backend Structure:



Responsibilities:

Controllers:

Handle HTTP requests.

Services:

Business Logic.

Models:

Database Interaction.

Routes:

API Endpoints.

Middlewares:

Authentication and Validation.

## 9. SECURITY DESIGN

Implemented Security Measures:

- Password Hashing
- Authentication Middleware
- Protected Routes
- Input Validation
- Tenant Data Isolation

Future Enhancements:

- Google Authentication
- Role-Based Permissions
- Multi-Factor Authentication

## **10. CONCLUSION**

The BizFlow design follows modern software engineering principles with a modular, scalable, and maintainable architecture. The system successfully integrates inventory management, billing, payment tracking, dashboard analytics, and PDF generation into a unified platform. The multi-tenant design ensures security and scalability, while the layered architecture simplifies future enhancements and maintenance.

The design provides a strong foundation for future expansion into advanced analytics, mobile applications, cloud synchronization, and enterprise-level business management features.